

Types of Databases — Architecture, Indexing, Optimization & Use Cases v1

Eleven families, what each is physically doing, and the access pattern that decides between them
storage model · index structures · consistency & transactions · scaling · when not to add another one

1 · How to Read This — The Two Axes that Actually Separate Databases

What a database actually is
A system that **stores data durably**, finds it again quickly, and keeps it correct while many people change it at once. Every family below makes the same three promises — they differ in what they optimise and what they give up.

Axis 1 — the storage model

Row-oriented	All of a record's fields stored together. Cheap to read or write one whole record. Relational, document, wide-column.
Column-oriented	All values of one field stored together. Cheap to scan a few fields over very many records. Analytical / OLAP.
Log-structured	Append, never update in place; merge later. Very fast writes, read amplification. LSM engines.
Graph-native	Edges stored as direct pointers, so traversal does not need an index lookup per hop.
Vector	High-dimensional embeddings with approximate nearest-neighbour search.

Axis 2 — the distribution model

- Single-node — one machine, strong consistency, simplest. Scales up, not out.
- Replicated — copies for availability and read scale; writes still funnel to a leader.
- Partitioned / sharded — data split by key across nodes. Scales writes, and costs you cross-shard joins and transactions.
- Distributed SQL — sharded and transactional, via consensus. Newer, and it removes the old "SQL cannot scale out" trade-off.

The question that decides it

Not "how much data?" — but "what does a typical read look like?" A ten-million-row table scanned by wide aggregates belongs somewhere completely different from a ten-billion-row table read only by primary key. Volume is a capacity question; access pattern is the design question.

If the dominant access is...	Then
Fetch or update one record by id	Relational, document, or key-value
Query by many different attributes, with joins	Relational
Aggregate a few columns over millions of rows	Columnar / OLAP
Write enormously, read by known key	Wide-column
Follow relationships several hops deep	Graph
Append timestamped points, query by range	Time series
Rank documents by relevance to text	Search
Find semantically similar items	Vector
Sub-millisecond lookup of transient state	In-memory key-value

Most systems get this wrong in one specific way: they pick the database first, then bend the access patterns to fit it. Write down the five queries that must be fast before choosing — and if you cannot list them, that is the finding, not the database.

Start relational, and make the second store earn its place
PostgreSQL competently handles relational, JSON documents, full-text search, geospatial, key-value, and vectors (via extensions), and TimescaleDB adds time series. For a large share of systems that is the entire data layer.

Every additional store has a standing cost that a benchmark never shows: another thing to back up, restore, patch, monitor, secure, upgrade and be paged for — plus dual writes, or a pipeline, and the consistency questions that come with either.

When a second store is genuinely justified

- The access pattern is fundamentally different, not merely slow — analytics beside OLTP, or ANN search
- You have measured the primary store failing at it, with the obvious indexing and tuning already done
- The scale is real and current, not projected
- Someone will own its operations, on call, at 3am

"Polyglot persistence" is a description, not a goal. Large systems end up with several stores because different subsystems genuinely differ. Starting there, before any of them is justified, buys all the operational cost of the end state and none of the benefit.

A note on names. "NoSQL" originally meant non-relational, not "not only SQL" — that reading came later. It groups families that have almost nothing in common with each other, so it is a poor basis for a decision.

2 · The Eleven Families at a Glance

Family	Stores	Primary index	Scales by	Consistency	Strong for	what each physically stores, how it finds things, how it scales, and what it is poor at
Relational (OLTP)	Rows in typed tables, fixed schema, foreign keys	B+ tree	Vertical + read replicas	Strong, ACID, isolation levels	Joins, ad-hoc queries, transactions, integrity	
Distributed SQL	Same relational model, sharded across nodes	B+ tree / LSM + Raft	Horizontal, transparently	Strong, distributed via consensus	Relational semantics and horizontal write scale; multi-region	
Document	Self-contained JSON/BSON documents, flexible schema	B+ tree on _id	Sharding by key	Tunable; multi-doc ACID available	Evolving schemas, aggregate-shaped data, fast development	
Key-value	Opaque value under a key	Hash	Consistent hashing	Varies — single-node strong to tunable	Caching, sessions, counters, rate limits, very fast simple access	
Wide-column column-family	Rows partitioned by key, each with many dynamic columns — stored row-wise	Partition key + clustering key, LSM	Horizontal, linear, no leader	Tunable per query (quorum)	Massive write throughput, time-ordered reads by known key, multi-region	
Columnar / OLAP	Values of one column stored contiguously — a different thing entirely; see panel 6	Sort key + zone maps, no per-row index	Horizontal; storage/compute split	Usually eventual; batch loaded	Aggregating few columns over billions of rows; compression	
Graph	Nodes, edges and properties; edges as direct pointers	Index-free adjacency + node/edge indexes	Hard — traversals resist partitioning	ACID in native engines	Multi-hop relationships, path finding, fraud rings, recommendations	
Time series	Timestamped points grouped by series, heavily compressed	Time + tag index	Partition by time and series	Eventual; append-mostly	Metrics, sensors, ingest at scale, downsampling and retention	
Search	Inverted index from terms to documents, plus the documents	Inverted index	Sharding + replicas	Near-real-time, eventual	Full-text relevance, faceting, fuzzy matching, log analytics	
Vector	High-dimensional embeddings + metadata	HNSW / IVF (approximate)	Sharding by vector space	Eventual	Semantic similarity, RAG retrieval, recommendations, image search	
Embedded	An in-process library, not a server — file or memory backed	B+ tree (SQLite) / columnar (DuckDB)	Does not — it runs inside your process	ACID (SQLite), single writer	Local and edge apps, tests, analytics on a laptop, CLI tools	

Two entries here are usually missing from a chart like this, and both matter. Distributed SQL is why "relational cannot scale horizontally" is no longer simply true. And SQLite is among the most widely deployed database engines in the world — it is in phones, browsers and applications everywhere, and it is frequently the right answer for local state.

3 · Relational & Distributed SQL

the default, and the reason its main limitation is now negotiable

Architecture

```
client → connection pool → parser → planner/optimizer
                                     + cost-based
transaction manager → buffer cache
                                     +
storage engine → data files → index files
                                     + WAL (write-ahead log, durability)
```

The WAL is the part worth understanding. Changes are written to a sequential log before the data files, which is what makes crash recovery, point-in-time restore and replication all possible from one mechanism.

Indexes available

Type	Use
B+ tree	The default. Equality, ranges, ordering, prefix matching
Hash	Equality only, no ranges
GIN	Multi-value columns — JSONB, arrays, full-text
GIST / SP-GIST	Geometric, geospatial, nearest-neighbour
BRIN	Very large naturally-ordered tables; tiny index, coarse skipping
Partial / covering	Index a subset, or answer from the index without touching the row

Composite index column order is not arbitrary. An index on (a,b) serves queries filtering on a, or on a **AND** b — but not on b alone. Most "the index isn't being used" problems are this.

Transaction isolation — usually omitted, and it matters

Level	Permits
Read uncommitted	Dirty reads. Effectively unused
Read committed	Non-repeatable reads, phantoms. The default in PostgreSQL and Oracle
Repeatable read	Phantoms (in the standard); write skew. MySQL/InnoDB default
Serializable	Nothing — behaves as if transactions ran one at a time. Costs throughput

Read committed does not protect a read-modify-write. Two concurrent "check the balance, then deduct" transactions can both read the same balance. You need an explicit lock, an atomic update, a version column, or serializable isolation — the default will not save you.

Optimisation, in the order that pays

- Read the query plan before changing anything. Most tuning is guesswork otherwise
- Index what you filter, join and sort on — and remove indexes nothing uses; each one slows every write
- Keep statistics current: a stale planner picks bad join orders
- Connection pooling — connections are expensive, and an unpooled app exhausts the server first
- Partition large tables by time or key so old data can be pruned or dropped cheaply
- Read replicas for read scale — and accept the replication lag they introduce

When single-node writes genuinely run out, distributed SQL keeps the relational model, joins and transactions while sharding underneath via consensus. It is a real option now — at the cost of higher write latency, since a commit needs agreement across nodes, and considerably more to operate.

4 · Document

aggregate-shaped data

The model
Each document is self-contained — the order, its lines and its address in one record. Read the whole thing in one operation, no join. Schema is enforced by the application, or optionally by the database.

```
{ _id: "01123", customer: "c_44",
  status: "shipped",
  lines: [ { sku: "A1", qty:2 }, { sku: "B7", qty:1 } ],
  address: { city: "Bengaluru", pin: "560011" } }
```

The design rule. Embed what is read together and changes together; reference what is shared or grows without bound. An array that grows for ever inside one document is the classic failure — documents have a size limit, and rewriting a large one on every append is expensive.

Indexing & tuning

- B+ tree on _id by default; add compound, text, geospatial and TTL indexes as needed
- Compound index order follows the same rule as SQL — equality fields, then sort, then range
- Shard key choice is effectively permanent — pick for even distribution and for query routing
- Covered queries answer from the index alone

On transactions — a common out-of-date claim. Multi-document ACID transactions have been supported since MongoDB 4.0 for replica sets, and 4.2 extended them across sharded clusters. They are no longer a missing capability. They do carry a cost, so the guidance is to model so that most operations touch a single document — not to assume transactions are unavailable.

Where it goes wrong, using it for data that is genuinely relational. If you find yourself joining three collections in application code and keeping duplicated fields in sync by hand, you have rebuilt a relational database without the guarantees.

5 · Key-Value & In-Memory

the simplest contract

One operation: get (key) / put (key, value). The value is opaque to the database, so there is nothing to query by except the key — which is exactly why it is fast.

Two quite different things

Kind	Character
In-memory	Sub-millisecond, data in RAM. Durability optional and a trade-off. For cache, sessions, counters, locks, rate limits, queues
Persistent, distributed	Disk-backed, horizontally scaled, durable. A system of record for key-addressed data

Redis is no longer only key-value. It has lists, sets, sorted sets, hashes, streams, geospatial and pub/sub. Sorted sets alone cover leaderboards, rate limiting and priority queues — most teams use a fraction of what it there.

Practical rules

- Design the key like an API: user:1183:cart — a consistent scheme is the only structure you get
- Always set a TTL to cache entries; unbounded keys become an eviction incident
- Know the eviction policy before memory fills, not after
- Keep values small — multi-megabyte values block the server and saturate the network
- Consistent hashing with virtual nodes so adding a node moves only its share of keys

The cache failure modes that cause outages: a stampede when many keys expire at once (fitter the TTIs), a hot key saturating one node, and cold-start after a restart — when the database receives the full load it has not been sized for.

6 · Wide-Column vs Columnar — Two Different Things with Similar Names

the most common confusion in database taxonomy, and choosing the wrong one is expensive

They are not variations of one idea. "Wide-column" (or column-family) describes a flexible schema: each row may hold different columns. "Columnar" describes a physical layout: values of one column stored contiguously on disk. Cassandra is the first and not the second — within a partition its data is laid out row-wise.

```
// WIDE-COLUMN (Cassandra) — row-wise on disk
partition "sensor_9":
[ts=10:00 | temp=21 | hum=40 ]
[ts=10:01 | temp=22 | hum=41 | batt=97 ]  ← extra col
one partition read = one sequential read

// COLUMNAR (ClickHouse) — column-wise on disk
ts: [ 10:00, 10:01, 10:02, ... ]
temp: [ 21, 22, 23, ... ]  ← compresses hard
hum: [ 40, 41, 42, ... ]
SUM(temp) reads only the temp column
```

Why the layout decides everything

- Point read by key — row layouts win decisively; the whole record is in one place
- Aggregate over millions of rows — column layouts win decisively; only the named columns are read, and they compress far better because adjacent values share a type
- Single-row update — cheap in a row store, expensive in a column store, which is built for batch loads

Cardinality is the failure mode. Every distinct tag combination is a separate series. Putting a user id, request id or trace id in a tag creates millions of series and will exhaust memory — this is the single most common way a metrics system falls over, and it arrives suddenly.

Choosing

Prometheus	Full-bless monitoring, alerting; short local retention
TimescaleDB	PostgreSQL extension — full SQL, and joins with relational data
InfluxDB	Purpose-built, push-based ingestion
ClickHouse	Often excellent for this too, when you also want analytics

If you already run PostgreSQL, Timescale is usually the lowest-friction option — it keeps SQL, joins against your relational tables, and one system to operate.

	Wide-column	Columnar / OLAP
Examples	Cassandra, ScyllaDB, HBase, Bigtable	
Built for	Write throughput and known-key reads	Scanning and aggregating
Query shape	Must include the partition key	Arbitrary SQL over any columns
Engine	LSM tree	Column segments + sort keys, zone maps
Joins	Avoided by design; denormalise instead	Supported, though wide tables are preferred
Updates	Write a new version; compaction resolves	Batch rewrite; single-row updates are awkward
Design driver	Model per query. One table per access pattern is normal and correct	Model per fact. Sort order determines what can be skipped

The wide-column rule that surprises people. You design tables from your queries, not from your entities — and duplicating the same data into several tables with different partition keys is the intended practice, not a mistake. There is no ad-hoc query escape hatch later.

The columnar rule that decides performance. Statistics can only skip a block if the values in it are clustered. Write rows in random order and every block's min/max spans the whole domain, so nothing is ever skipped — the statistics are present, correct and useless. Sort order at write time matters more than any query tuning.

Choosing wrongly is not a tuning problem. Cassandra will not become an analytics engine, and ClickHouse will not become a transactional store. Both failures usually surface only at production scale.

relevance, not matching

A database answers "which rows match?". A search engine answers "which documents are most relevant, ranked?" — a different question, and the inverted index is what makes it fast.

```
// inverted index: term → documents containing it
"database" = [d1, d7, d9, d12 ...]
"indexing" = [d7, d9, d41 ...]
// intersect the lists, then score by relevance
```

The pipeline that decides quality

- Analysis — tokenise, lowercase, remove stopwords, stem or lemmatise. Applied identically at index and query time, or results are silently wrong
- Scoring — BM25 is the modern default; it improves on TF-IDF by saturating term frequency and normalising for document length
- Fuzzy matching — edit distance for typos; n-grams for partial words; phonetic for names
- Faceting and filters — counts by category alongside results

Hybrid search is now the norm. Keyword search finds exact terms, product codes and names; vector search finds meaning. Running both and fusing the ranked lists beats either alone, particularly on short queries where semantics are ambiguous.

Do not make it the system of record. Search indexes are near-real-time, eventually consistent, and rebuildable by design. Keep the authoritative copy in a database and project into the index — losing the index should mean a rebuild, not data loss.

PostgreSQL full-text search is genuinely capable and removes an entire system. Move to a dedicated engine when you need relevance tuning, faceting at scale, or search-specific analytics.

7 · Graph

when relationships are the data

Nodes, edges and properties, where an edge is a direct pointer rather than a foreign key to be looked up. Traversing a hop is a pointer dereference, not an index seek — which is what "index-free adjacency" means.

The comparison that justifies it. A three-hop relationship query in SQL is three joins; five hops is five, and cost grows with table size at every step. In a native graph engine, traversal cost depends on how many edges you actually walk — not on how large the graph is.

Use it when

- Depth is variable or unbounded — "anyone connected to X within 4 hops"
- The relationship carries meaning — type, weight, direction, time
- Path itself is the answer — routing, dependency chains, provenance, impact analysis
- Fraud rings, identity resolution, knowledge graphs, recommendations, supply chains, network topology, access control

Algorithms you will meet

BFS / DFS	Reachability and traversal
Dijkstra / A*	Shortest and cheapest path
PageRank	Influence and importance
Community detection	Clusters, rings, segments
Similarity	Jaccard, cosine over neighbourhoods

The honest limitation. Graphs resist partitioning — a traversal that crosses machines is far more expensive than one that does not, and there is no partition key that makes all traversals local. Graph databases scale up more readily than out.

Not every relationship needs one. Two joins in an existing relational database beat operating a second system. The threshold is variable-depth traversal, not the presence of foreign keys.

8 · Time Series

append-mostly, ordered

Every record is a timestamp, a series identity (tags) and a value. The workload is extremely predictable — heavy append, query by time range, almost never update — and the engines exploit exactly that.

What the specialisation buys

- Compression by an order of magnitude — delta and delta-of-delta on timestamps, XOR on floats. Adjacent values are similar, so there is a great deal of structure to exploit
- Time partitioning — old chunks are dropped, not deleted row by row
- Continuous aggregates — hourly and daily rollups maintained as data arrives
- Retention and downsampling policies as first-class configuration
- Time-aware functions — gap filling, interpolation, rate, time buckets

Cardinality is the failure mode. Every distinct tag combination is a separate series. Putting a user id, request id or trace id in a tag creates millions of series and will exhaust memory — this is the single most common way a metrics system falls over, and it arrives suddenly.

Choosing

Prometheus	Full-bless monitoring, alerting; short local retention
TimescaleDB	PostgreSQL extension — full SQL, and joins with relational data
InfluxDB	Purpose-built, push-based ingestion
ClickHouse	Often excellent for this too, when you also want analytics

If you already run PostgreSQL, Timescale is usually the lowest-friction option — it keeps SQL, joins against your relational tables, and one system to operate.

9 · Search

relevance, not matching

Read, write and space amplification
Every engine picks a point on a three-way trade-off, and the choice explains most of its behaviour:

	B+ tree	LSM tree
Writes	Slower — update in place, random I/O	Faster — sequential appends
Reads	Predictable, one path	May check several files, bloom filters mitigate
Space	Fragmentation from in-place updates	Old versions retained until compaction
Background work	Vacuum / defragmentation	Compaction — significant, and it competes with your traffic

This is why Cassandra writes so fast and why PostgreSQL reads so predictably. Neither is better; they occupy different points on the same trade-off.

Rules that apply to every database

- An index speeds reads and slows every write. Unused indexes are pure cost — find and drop them
- Composite order matters — leading column first, and only prefixes of the index are usable
- Wrapping a column in a function disables its index unless you built a matching expression index
- Covering indexes answer from the index alone and avoid touching the row entirely
- Low-cardinality columns rarely deserve their own index — a boolean index seldom helps
- Measure with the query plan, never by assumption

Sharding, partitioning and replication are three different things. Partitioning splits a table within one database. Sharding splits data across machines. Replication copies the same data for availability and read scale. They are frequently combined and frequently confused.

12 · Consistency, Transactions & Distribution

the guarantees each family gives up, stated plainly

ACID and BASE

ACID	Meaning
Atomicity	All of a transaction happens, or none of it
Consistency	Constraints hold before and after
Isolation	Concurrent transactions do not corrupt each other — to a stated degree
Durability	Once committed, it survives a crash

BASE — basically available, soft state, eventual consistency. Not a weaker implementation of ACID but a different bargain: accept temporary disagreement between replicas in exchange for availability and write scale. The question is never which is better, but which your use case can tolerate.

CAP and the more useful version

- CAP — during a network partition, choose consistency or availability. It says nothing about normal operation, which is where the popular reading goes wrong
- PACELC — and else, when there is no partition, choose latency or consistency. This is the trade-off you actually make every day

"Eventually consistent" needs a number. Eventually is milliseconds in one system and minutes in another under load. Ask what the replication lag is at p99, and what the application does when a user writes and immediately reads a stale value.

Consistency models, strongest first

Strict / linearizable	Reads always see the latest committed write. Costs coordination
Sequential	Everyone sees operations in the same order
Causal	Related operations are ordered; unrelated ones may differ
Read-your-writes	You see your own updates. Often the minimum users notice
Monotonic reads	Time never appears to go backwards
Eventual	Replicas converge if writes stop. The weakest useful guarantee

Quorums make this tunable. With N replicas, reading from R and writing to W, R + W > N guarantees a read sees the latest write. Cassandra and DynamoDB expose this per query, so consistency becomes a per-operation decision rather than a database-wide one.

Distribution mechanics

- Leader-follower — one writer, many readers. Simple, and the leader is the write bottleneck
- Multi-leader — write anywhere; you must now resolve conflicts
- Leaderless — quorum reads and writes, with read repair and anti-entropy
- Consensus (Raft/Paxos) — how distributed SQL keeps strong consistency while sharded, at the cost of a round trip per commit

Replication is not a backup. It faithfully copies a mistaken DELETE to every replica in milliseconds. Only a backup — tested by restoring it — protects against a bad deployment or a wrong command.

What each family actually gives you

Family	Transaction reality
Relational	Full ACID across rows and tables, with a choice of isolation level
Distributed SQL	ACID across shards via consensus; commits cost a network round trip
Document	Always atomic within one document; multi-document ACID supported, and best avoided by modelling
Key-value	Atomic per key. Redis has scripts and transactions; DynamoDB has conditional and multi-item writes
Wide-column	Atomic per partition. No multi-row transactions — this is a design constraint, not a gap
Columnar	Batch-oriented; some now offer table-level ACID via a table format
Graph	ACID in native engines, across a traversal
Time series / Search / Vector	Append-oriented, eventually consistent. Do not depend on them for transactional correctness

Across services, not just rows

- Two-phase commit — atomic, blocking, and it needs a coordinator you must keep alive. Rare in modern designs
- Saga — a sequence of local transactions with compensating actions. Gives up isolation, so intermediate states are visible and must be modelled
- Outbox — write the state change and its event in one local transaction, then publish. The standard way to avoid dual writes

The practical hierarchy. Keep data that must change together in one database and one transaction wherever you can. Reach for a saga only when a boundary genuinely forces it — the coordination is far more expensive than it appears in a diagram.

13 · Choosing, and the Checks before you Commit

the decision is reversible only while the data is small

- The five queries that must be fast are written down before any product is chosen
- Read/write ratio known, and peak rather than average
- Data volume and growth estimated over 12–18 months
- Consistency requirement stated per operation, not for the whole system
- Latency target at p99, not average
- Access pattern matched to storage model — point reads vs scans vs traversals
- Checked whether the existing design does this adequately
 - Partition or shard key chosen for distribution and for query routing
 - Shard key treated as permanent — changing it is a migration, not a setting
 - Hot-partition risk checked against the real key distribution

- Indexes cover the filters, joins and sorts actually used
- Unused indexes identified and dropped
- Query plans reviewed for the top queries, not assumed
- Statistics refreshed after bulk loads
- Connection pooling in place and sized
- Isolation level understood, and read-modify-write paths protected explicitly
- Replication lag measured, and state-read behaviour defined
- Backups automated, offsite, encrypted — and restore tested and timed
- RT0 and RPO agreed with whoever owns the tolerance
- Schema migration path is expand → migrate → contract, tested at production data size

- Time-series cardinality bounded — no ids in tags
- Cache TTIs jittered; eviction policy chosen deliberately
- ANN recall measured against exact search on a sample
- Search index treated as rebuildable, never as the source of truth
- Monitoring covers slow queries, lock waits, replication lag, disk and connection saturation
- Encryption in transit and at rest; access control and audit in place
- Someone owns its operations — upgrades, patching, capacity and on-call
- Team can operate it, or the learning cost is budgeted
- Licence and cost model understood at projected scale
- Exit path considered — how data would leave if this choice proves wrong

The one that summarises the sheet: name the access pattern before naming the database. Nearly every expensive migration traces back to a system chosen for its category — or its popularity — rather than for the shape of the queries it would actually receive.